What is Hadoop?

Hadoop is an open-source framework developed by the Apache Software Foundation for storing and processing large datasets (Big Data) in a distributed computing environment.

It allows data to be stored across clusters of computers and processed in parallel, making it highly scalable, fault-tolerant, and cost-effective.

E-commerce (Amazon, Flipkart, eBay)

•Hadoop stores massive data like **customer purchases, browsing history, and clickstreams**.

•Used for **recommendation systems**

Social Media (Facebook, Twitter, LinkedIn)

•Handles **billions of posts, likes, shares, and comments daily**.

•Hadoop processes this unstructured data to detect **trending topics, targeted ads, and spam detection**.

Why Hadoop

Key Consideration :

" Its capability to handle massive amounts of data , different categories of data – fairly quickly.

Other Considerations:

1.Low Cost:

Hadoop is an open source framework and uses commodity software to store enormous quantities of data

2. Computing Power:

Hadoop is based on distributed computing model which processes large volume of data very quickly and fairly .The more number of computing nodes, the more power

3. Scalability:

Can easily add nodes as the system grows and requires much less adinistration

4. Storage flexibility:

One can store unstructured data like images, videos and free-form text

5. Inherent data protection:

Protects data against hardware failure

Why not RDBMS?

RDBMS verses Hadoop

Parameters	RDBMS	Hadoop
System	Relational Database Management System	Node based flat structure
Data	Suitable for structured data	Suitable for variety of data such as XML, JSON, text based flat file formats etc
Processing	OLTP	Analytical, Big Data Processing
Choice	When the data needs consistent relationship	Big data processing which does not require any consistent relationships between data
Processor	Needs expensive h/w or high end processors to store huge volume of data	In a Hadoop cluster, a node requires only a processor, a network card and few hardware drives

RDBMS vs. Hadoop

	RDBMS	HADOOP
Data size	Gigabytes *(Terabytes)*	Petabytes *(Hexabytes)*
Access	Interactive and Batch	Batch
Updates	Read / Write many times	Write once, Read many times
Structure	Static Schema	Dynamic Schema
Integrity	High (ACID)	Low
Scaling	Nonlinear 	Linear 

Hadoop v/s SQL

Aspect	Hadoop	SQL (Traditional Databases)
Definition	An **open-source framework** for distributed storage and parallel processing of large datasets.	A **language (Structured Query Language)** used to manage and query relational databases (RDBMS).
Data Type Support	Works with **structured, semi-structured, and unstructured data** (text, video, logs, images, etc.).	Primarily works with **structured data** stored in rows and columns.
Storage System	Uses **HDFS (Hadoop Distributed File System)** to store data across multiple nodes.	Uses **tables in a relational database** (like MySQL, Oracle, PostgreSQL).
Processing Model	Processes data using **MapReduce** (batch) or **Spark** (real-time, in-memory).	Processes queries using **SQL engine** within a single server or limited cluster.
Scalability	**Highly scalable** – can add commodity servers (nodes) easily.	Scaling is **vertical** (upgrading CPU, RAM of a single server), which is limited and costly.

Distributed Computing Challenges

1.Hardware Failure

In a distributed system, several servers are networked together

There may be a possibility of failure

Solution: This problem can be solved using **Replication Factor(RF)**

Replication Factor is used multiple copies of the same data block are stored on different servers.

Even if one server fails, the data is safe on another.

2. How to Process This Gigantic Store of Data ?

•In distributed systems, data is scattered across many machines. Processing requires combining (integrating) data from all servers, which is very complex.

Solution: Hadoop's MapReduce programming model

•**Map:** Breaks the problem into smaller tasks and processes them on each machine where the data resides.

•**Reduce:** Collects and combines the results into the final output.

Hadoop Overview

Hadoop is a open source framework to store and process massive amount of data in a distributed fashion on large clusters of commodity hardware.

Hadoop accomplishes two tasks:

1. Massive Data Storage
2. Faster Data Processing

Key aspects of Hadoop are:

1.Open source Software: Free to download, use and contribute to

2.Framework: Programs, tools etc for apllication developement are provided

- **Distributed**: Divides and stores data across multiple computers
- **Massive Storage**: Stores large amount of data across nodes of low cost commodity hardware
- **Faster Processing**: Data is processed in parallel, yielding quick response

Features of Hadoop

Flexibility in Data Processing

Hadoop can handle all types of data—whether **structured, semi-structured, unstructured, encoded, or formatted**. This makes it versatile for managing diverse datasets.

High Scalability

Hadoop provides excellent scalability, allowing **new nodes to be added easily** as data volume or processing requirements increase, without needing to modify existing systems or applications.

Fault Tolerance

Data in Hadoop is stored in **HDFS (Hadoop Distributed File System)**, where it is automatically **replicated across multiple nodes**. Even if one or two nodes fail, the data remains safe and accessible from other copies, ensuring strong fault tolerance.

High-Speed Data Processing

Hadoop excels at batch processing of large-scale data by leveraging parallel processing.

Compared to single-threaded servers or mainframes, Hadoop can perform batch tasks up to 10 times faster.

Robust Ecosystem

Hadoop isn’t just one tool. It has an ecosystem with tools like

Hive (SQL-like queries), Pig (data flow scripts), HBase (NoSQL database),

Spark (real-time data processing), etc.

This makes it flexible for different business needs.

Example:Take Uber.

Uber uses Hadoop ecosystem tools to:Hive → Run queries on ride history.

Spark → Process real-time ride requests and driver availability.

HBase → Store trip and payment details quickly.

Cost-Effectiveness

By utilizing commodity hardware and enabling massively parallel computing,

Hadoop significantly lowers the cost per terabyte of storage.

Advantages of Hadoop

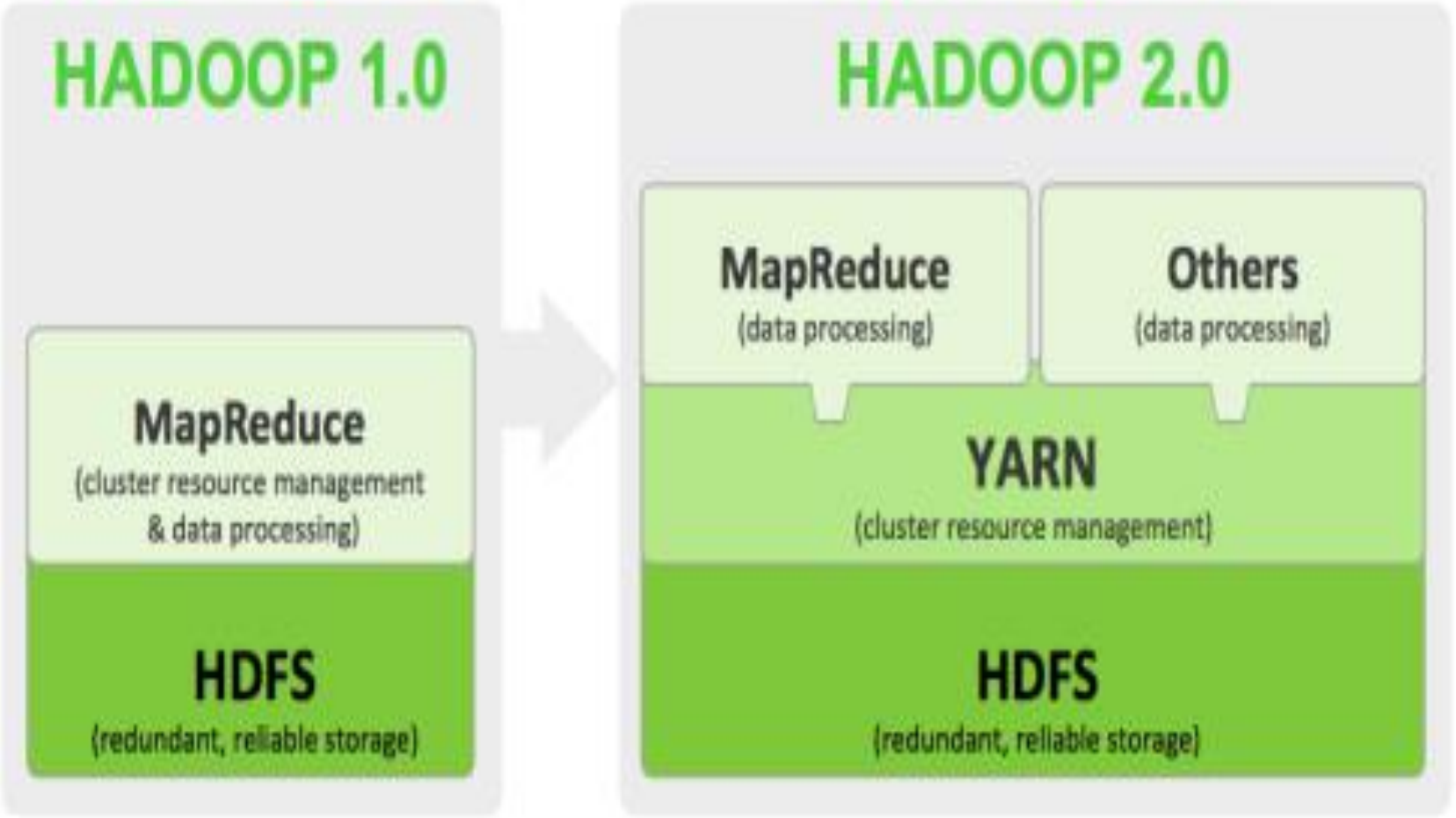
- Stores data in its native format **(schema-less)**
- It can store and distribute very large data sets across hundreds of inexpensive servers that operate in parallel **(Scalable)**
- It has much reduced cost/terabyte of storage and processing **(cost-effective)**
- It is fault-tolerant**(Resilient to failure)**
- It can work with all kinds of data.**(Flexibility)**
- The processing is extremely fast bcoz of 'move code to data' paradigm**(Fast)**

Versions of Hadoop

There are two versions:

1. Hadoop 1.0

2. Hadoop 2.0



Hadoop 1.0

It has two main parts

1. Data Storage framework:

- It is HDFS and is schema-less.
- It stores data files and these data files can be in any format.

2. Data Processing framework:

- It is a functional programming model and uses two functions to process the data.
- The MAP and the REDUCE
- The mappers take in a set of key-value pairs and generate intermediate data .
- The Reducers then act on this input to produce output data.
- These two funtions work in isolation from one another , thus enabling the processing to be
- highly distributed in a highly-parallel, fault-tolerant and scalable way.

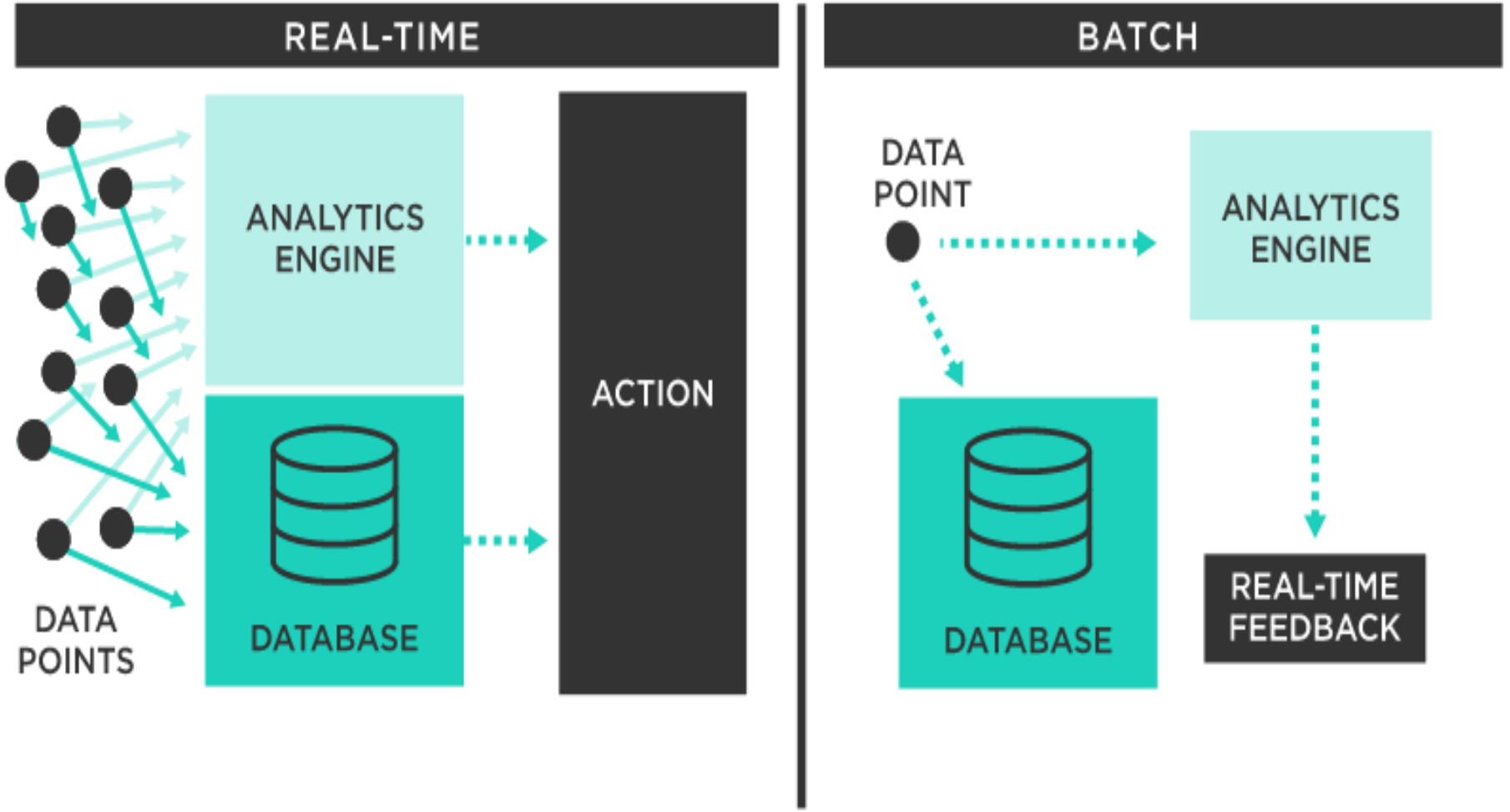
Limitaions of Hadoop 1.0

1. The requiremnt for MapReduce Programming expertise along with proficiency required in other programming languages (notably java)

2. It supported only Batch processing (Batch Processing system is an efficient way of processing large volumes of data. where a group of transactions is collected over a period of time.
Data is collected, entered, processed and then the batch results are produced.)

3. It is tightly , computationally coupled with MapReduce Programming which means users are left with two options

- Either rewrite their functionality in MapReduce so that so that it could be executed in Hadoop
- (or) Extract the data from HDFS and process it outside the Hadoop


REAL-TIME
ANALYTICS ENGINE
ACTION
DATA POINTS
DATABASE
BATCH
DATA POINT
ANALYTICS ENGINE
DATABASE
REAL-TIME FEEDBACK

Hadoop 2.0

- In Hadoop 2.0 , HDFS is the data storage framewok.
- However a new separate resouce management framework called

YARN(Yet Another Resource Negotiator) has been added.

- Any application capable of dividing itself into parallel task is supported by YARN.
- Here MapReduce programming expertise is no longer needed.
- It supports both batch processing and real time processing.

Hadoop 1.X	Hadoop 2.X
Limited to 4,000 nodes per cluster	Up to 10,000 nodes per cluster
'0' number of tasks in a cluster	'0' (Cluster Size)
Jobtracker bottleneck	Efficient cluster utilization - YARN
Has only one namespace for handling HDFS	Supports multiple namespace for handling HDFS
Map and Reduce slots are static	Not restricted to Java
Has only one job - to run MapReduce	Any application can integrate with Hadoop

Hadoop Components

HaDOOP Core Components:

1. HDFS (Hadoop distributed file system)

a. Storage component

b. Distributes data across several nodes

c. Natively redundant

2. Map Reduce

a. Computational framework

b. Splits a task across multiple nodes

c. Processes data in parallel

3. Hadoop ecosystem

Hadoop ecostsem are support projects to enhance the functionality of Hadoop core components.

Apache Hadoop Ecosystem

Ambari
Provisioning, Managing and Monitoring Hadoop Clusters

Sqoop Data Exchange

Flume Log Collector

Zookeeper Coordination

Oozie Workflow

Pig Scripting

Mahout Machine Learning

R Connectors Statistics

Hive SQL Query

Hbase Columnar Store

YARN Map Reduce v2
Distributed Processing Framework

HDFS
Hadoop Distributed File System

Hadoop Distributed File System (HDFS) -

➢The default big data storage layer for Apache Hadoop is HDFS.

➢HDFS is the “Secret Sauce” of Apache Hadoop components as users can dump huge datasets into HDFS and the data will sit there nicely until the user wants to leverage it for analysis.

➢HDFS component creates several replicas of the data block to be distributed across different clusters for reliable and quick data access.

➢HDFS comprises of 3 important components-***NameNode, DataNode and Secondary NameNod***

➢HDFS operates on a **Master-Slave architecture model** where the ***NameNode*** acts as the **master node** for keeping a track of the storage cluster and the ***DataNode*** acts as a **slave node** summing up to the various systems within a Hadoop cluster.

Data Access Components of Hadoop Ecosystem- Pig and Hive Pig-

Apache Pig is a convenient tools developed by Yahoo for analysing huge data sets efficiently

and easily.

It provides a high level data flow language Pig Latin that is optimized, extensible and easy to use

The most outstanding feature of Pig programs is that their structure is open to considerable

parallelization making it easy for handling large data sets.

Hive-

Hive developed by Facebook is a data warehouse built on top of Hadoop and provides a

simple language known as HiveQL similar to SQL for querying, data summarization and analysis

Hive makes querying faster through indexing.

Data Integration Components of Hadoop Ecosystem- Sqoop and Flume

Sqoop

Sqoop component is used for importing data from external sources into related Hadoop components like HDFS, HBase or Hive.

It can also be used for exporting data from Hadoop o other external structured data stores.

Sqoop parallelized data transfer, mitigates excessive loads, allows data imports, efficient data analysis and copies data quickly.

Flume

Flume component is used to gather and aggregate large amounts of data.

Apache Flume is used for collecting data from its origin and sending it back to the resting

Data Storage Component of Hadoop Ecosystem –HBase

HBase –

HBase is a column-oriented database that uses HDFS for underlying storage of data.

HBase supports random reads and also batch computations using MapReduce.

With HBase NoSQL database enterprise can create large tables with millions of rows and columns on hardware machine.

The best practice to use HBase is when there is a requirement for random ‘read or write’ access to big datasets.

Monitoring, Management and Orchestration(COORDINATION)

Components of Hadoop Ecosystem- Oozie and Zookeeper

Oozie-

Oozie is a workflow scheduler where the workflows are expressed as Directed Acyclic Graphs.

Oozie runs in a Java servlet container Tomcat and makes use of a database to store all the running workflow instances, their states ad variables along with the workflow definitions to manage Hadoop jobs (MapReduce, Sqoop, Pig and Hive).

The workflows in Oozie are executed based on data and time dependencies.

Zookeeper-Zookeeper is the king of coordination and provides simple, fast, reliable and ordered operational services for a Hadoop cluster.

Zookeeper is responsible for synchronization service, distributed configuration service and

Apache AMBARI or Call it the Elephant Rider

A Hadoop component, Ambari is a RESTful (representational state transfer)API (web based tool)

which provides easy to use web user interface for Hadoop management.

Ambari provides step-by-step wizard for installing Hadoop ecosystem services.

It is equipped with central management to start, stop and re-configure Hadoop services and it

facilitates the metrics collection, alert framework, which can monitor the health status of the

Hadoop cluster.

Apache MAHOUT

Mahout is an important Hadoop component for machine learning, this provides implementation

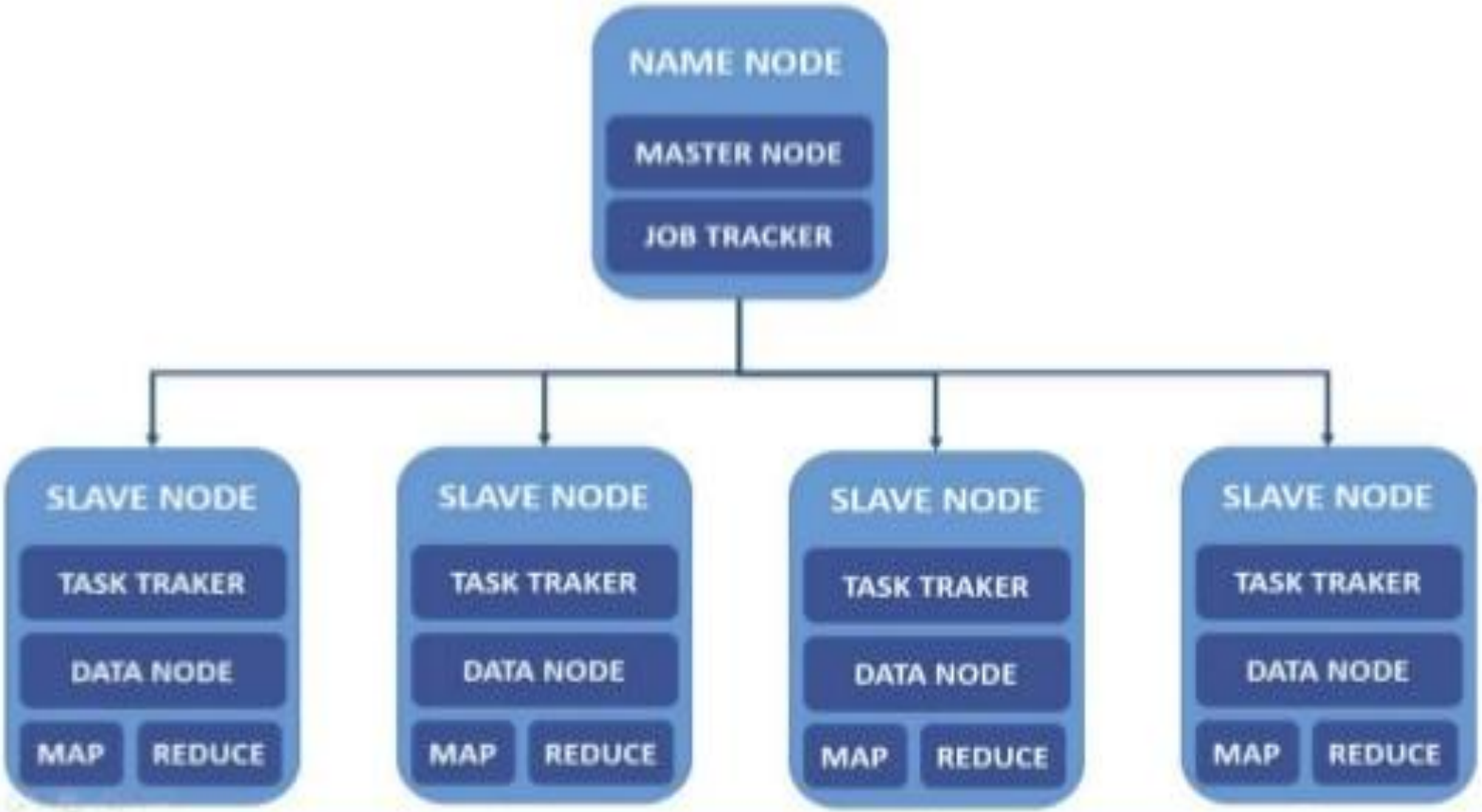
of various machine learning and data mining algorithms.

This Hadoop component helps with considering user behavior in providing suggestions,

categorizing the items to its respective group, classifying items based on the categorization and

supporting in implementation group mining or itemset mining, to determine items which appear in group.

HADOOP MASTER/SLAVE ARCHITECTURE



➢HDFS operates on a **Master-Slave architecture model** where the ***NameNode*** acts as the **master node** for keeping a track of the storage cluster and the ***DataNode*** acts as a **slave node** summing up to the various systems within a Hadoop cluster.

➢**Components of Master Node are:**

➢ **1. Master HDFS (Name node):**

Its main responsibility is partitioning the data storage across several nodes

It also keeps track of locations of data on Data Nodes

➢Hadoop follows a master slave architecture design for data storage and distributed data processing using HDFS and MapReduce respectively.

➢The master node for data storage is hadoop HDFS is the NameNode and the master node for parallel processing of data using Hadoop MapReduce is the Job Tracker.

➢The slave nodes in the hadoop architecture are the other machines in the Hadoop cluster which store data and perform complex computations.

➢Every slave node has a Task Tracker daemon and a DataNode that synchronizes the processes with the Job Tracker and NameNode respectively. In Hadoop architectural implementation the master or slave systems can be setup in the cloud or on-premise.

Key Aspects of HDFS

NameNode

- HDFS works in a master-slave/master-worker fashion.
- All the metadata related to HDFS including the information about data nodes, files stored on
- HDFS, and Replication, etc. are stored and maintained on the NameNode.
- A NameNode serves as the master and there is only one NameNode per cluster.

DataNode

- DataNode is the slave/worker node and holds the user data in the form of Data Blocks.
- There can be any number of DataNodes in a Hadoop Cluster.

Data Block

- A Data Block can be considered as the standard unit of data/files stored on HDFS.
- Each incoming file is broken into 64 MB by default.
- Any file larger than 64 MB is broken down into 64 MB blocks and all the blocks which make up a particular file are of the same size (64 MB) except for the last block which might be less than 64 MB depending upon the size of the file.

Replication

- Data blocks are replicated across different nodes in the cluster to ensure a high degree of fault tolerance.
- Replication enables the use of low cost commodity hardware for the storage of data.
- The number of replicas to be made/maintained is configurable at the cluster level as well as for each file.
- Based on the Replication Factor, each file (data block which forms each file) is replicated many times across different nodes in the cluster.

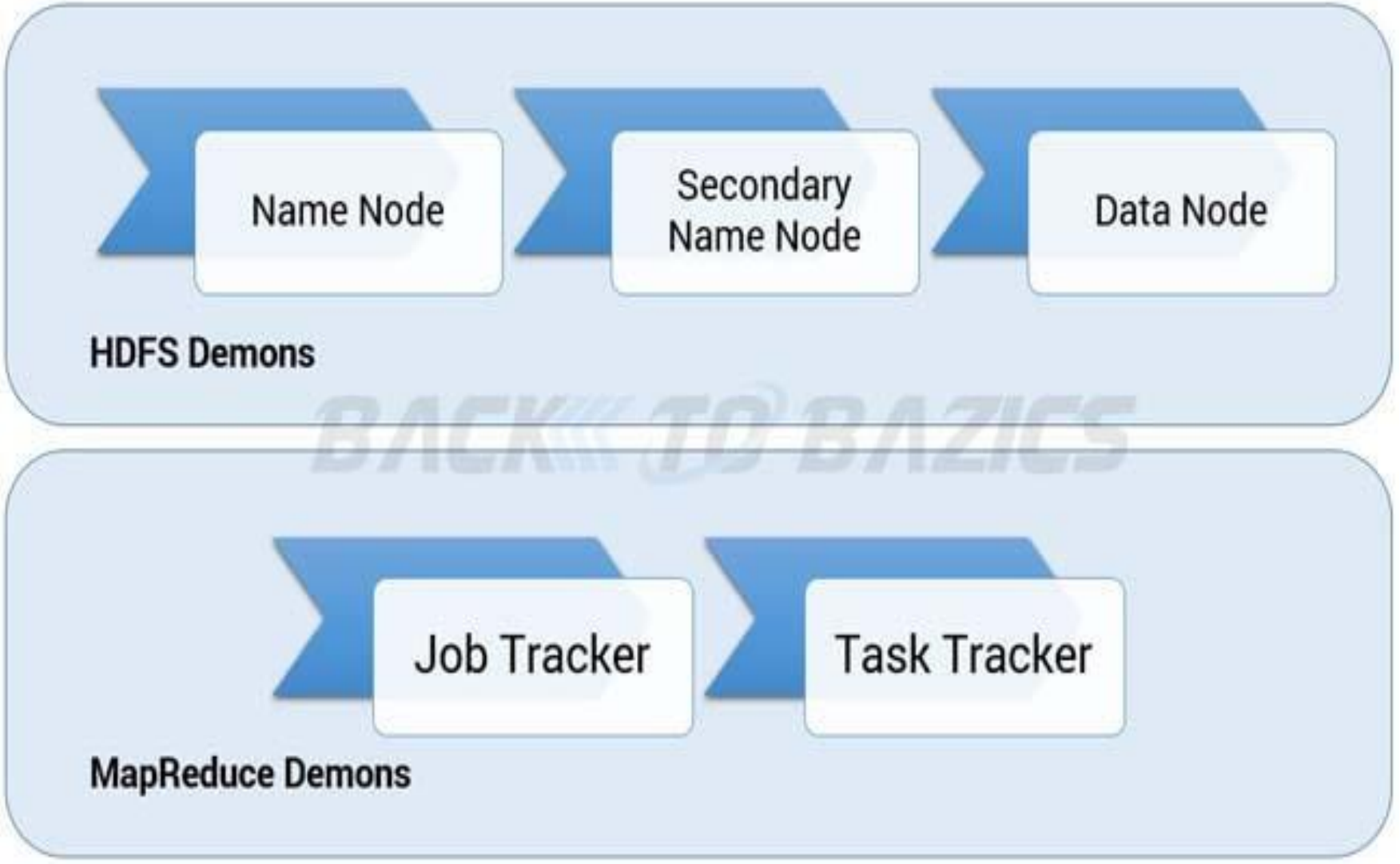
Rack Awareness

Data is replicated across different nodes in the cluster to ensure reliability/fault tolerance.

Replication of data blocks is done based on the location of the data node, so as to ensure high degree of fault tolerance.

For instance, one or two copies of data blocks are stored on the same rack, one copy is stored on a different rack within the same data center, one more block on a rack in a different data center, and so on.

Hadoop 1.x Architecture



HDFS Architecture in Hadoop 1.x mainly has 3 daemons which are Name Node, Secondary Name Node and Data Node.

Name Node

- There is only single instance of this process runs on a cluster and that is on a master node
- It is responsible for manage metadata about files distributed across the cluster
- It manages information like location of file blocks across cluster and it's permission
- This process reads all the metadata from a file named fsimage and keeps it in memory
- After this process is started, it updates metadata for newly added or removed files in RAM
- It periodically writes the changes in one file called edits as edit logs
- This process is a heart of HDFS, if it is down HDFS is not accessible any more

Secondary Name Node (Helper to Namenode)

- For this also, only single instance of this process runs on a cluster
- This process can run on a master node (for smaller clusters) or can run on a separate node (in larger clusters) depends on the size of the cluster
- One misinterpretation from name is “This is a backup Name Node” but IT IS NOT!!!!!
- It manages the metadata for the Name Node. In the sense, it reads the information written in edit logs (by Name Node) and creates an updated file of current cluster metadata
- Than it transfers that file back to Name Node so that fsimage file can be updated
- So, whenever Name Node daemon is restarted it can always find updated information in fsimage file

Data Node

- There are many instances of this process running on various slave nodes(referred as Data nodes)
- It is responsible for storing the individual file blocks on the slave nodes in Hadoop cluster
- Based on the replication factor, a single block is replicated in multiple slave nodes

(only if replication factor is > 1) to prevent the data loss

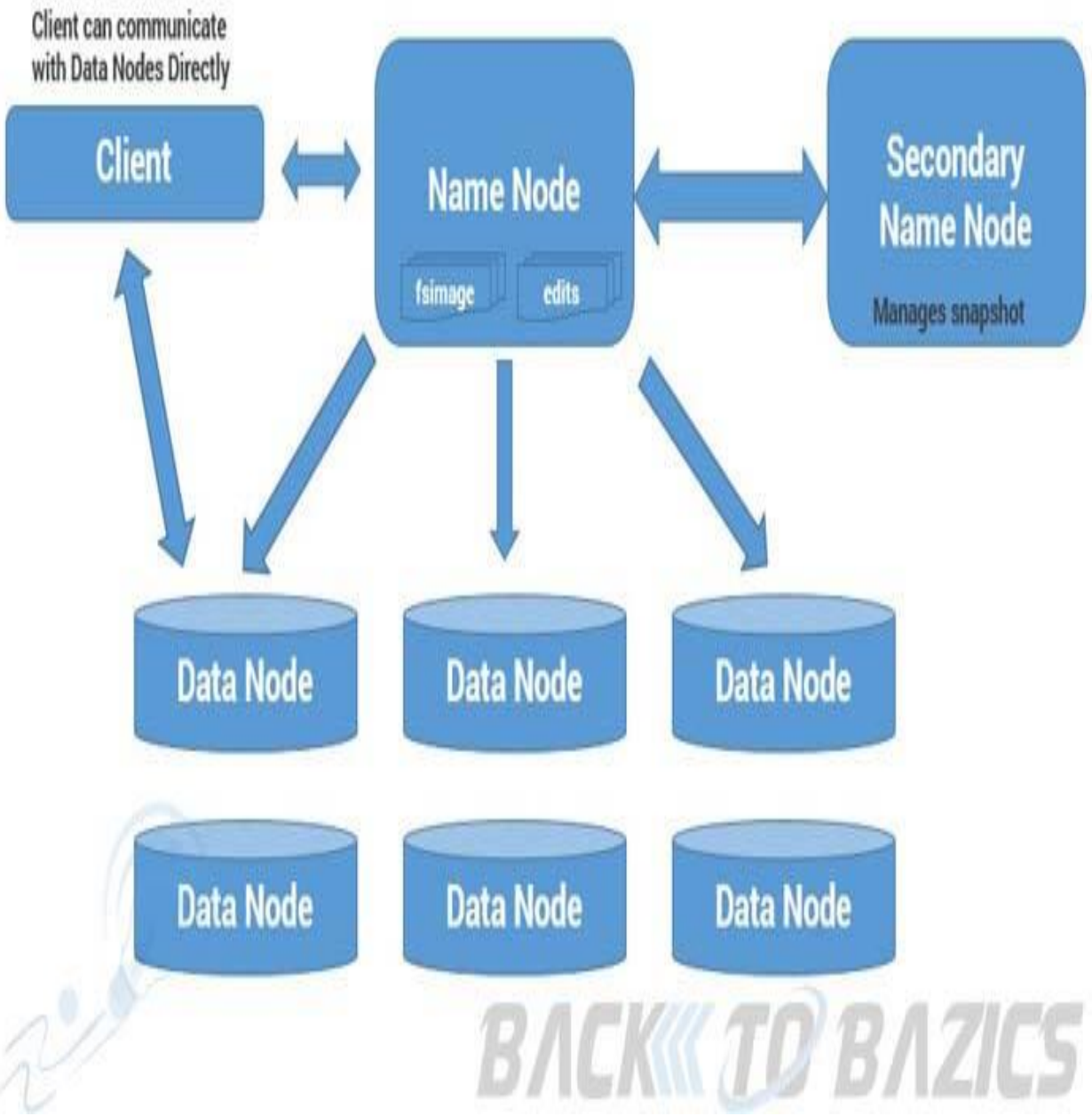
- Whenever required, this process handles the access to a data block by communicating with

Name Node

- This process periodically sends heart bits to Name Node to make Name Node aware that slave

process is running

- **Client** → The user/application that wants to store or read data.
- **NameNode** → The master that stores *metadata* (like a catalog of which file is stored where).
- **fsimage** → Snapshot of the file system.
- **edits** → Log of recent changes.
- **Secondary NameNode** → Helper that merges *fsimage* + *edits* to keep metadata updated.
- **DataNodes** → The actual storage servers where file blocks are saved.



HDFS master-slave/master-worker Working Model:

- NameNode servers as the master and each DataNode servers as a worker/slave.

- User data is stored on the local file system of DataNodes.

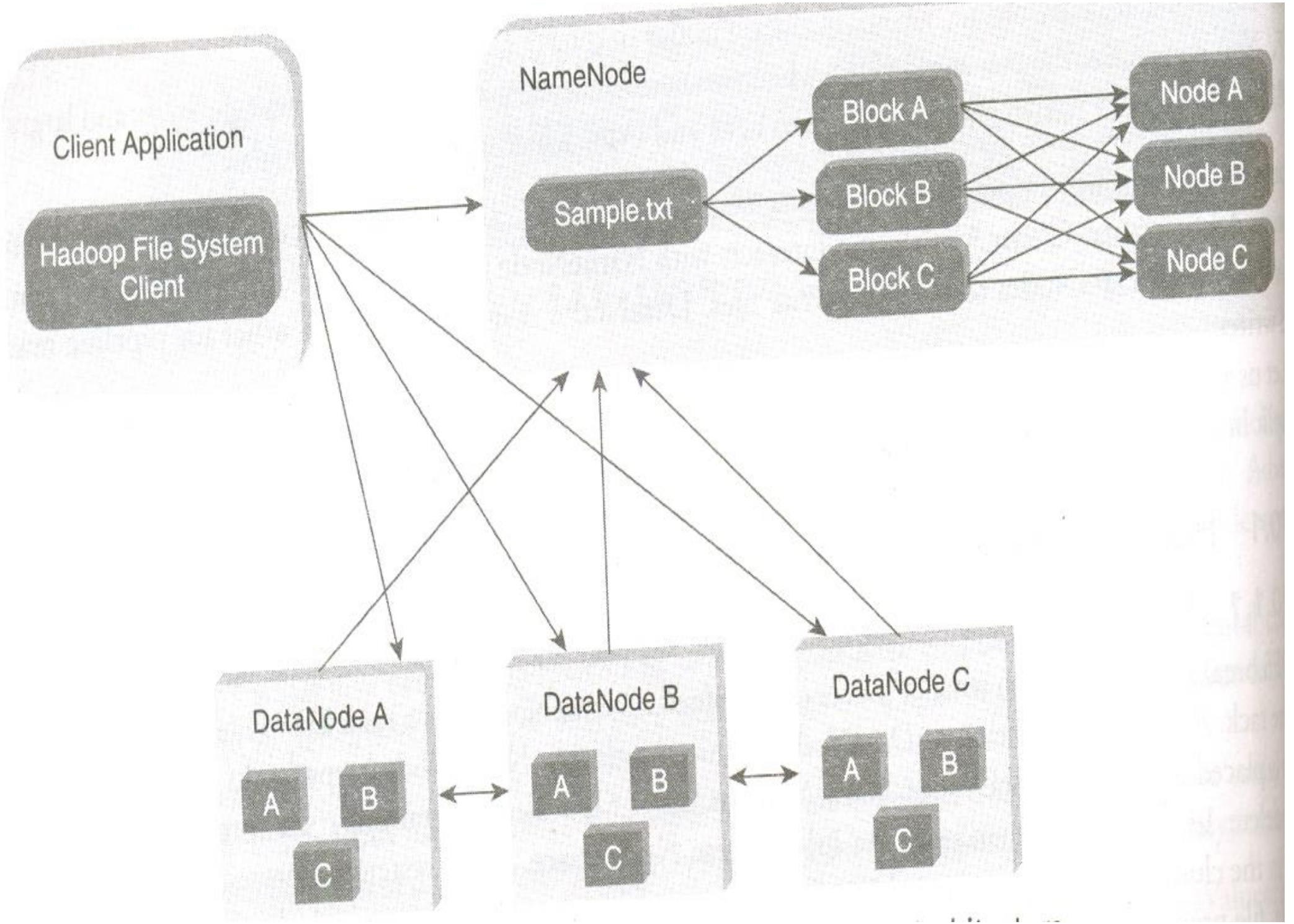
- If the NameNode goes down then the data in HDFS is non-usable as only the
- NameNode knows which blocks belong to which file, where each block located etc.

- NameNode can talk to all the live DataNodes and the live DataNodes can talk to each other.

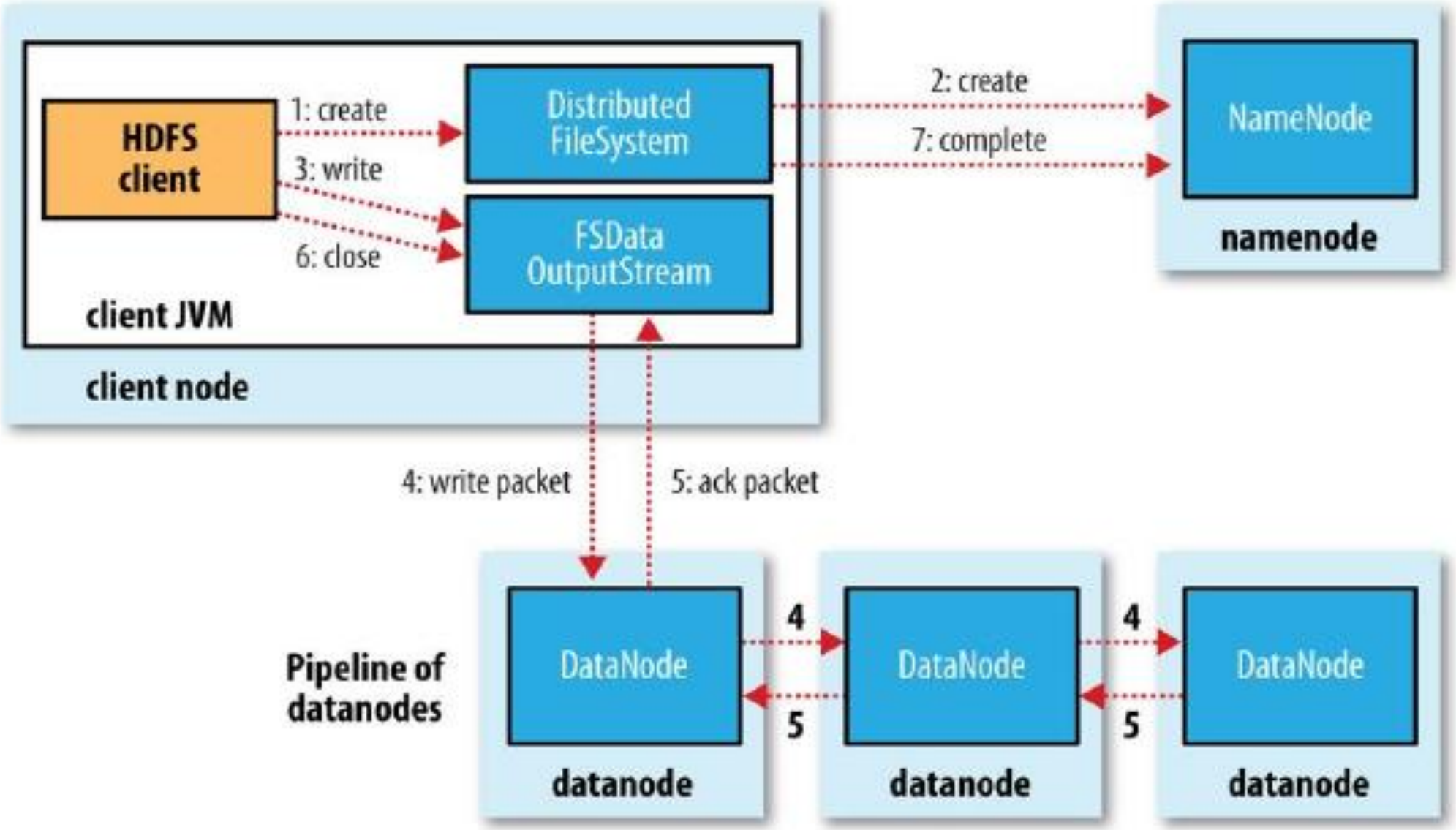
- There is also a Secondary NameNode which comes in handy when the Primary NameNode goes down.
- Secondary NameNode can be brought up to bring the cluster online.

- This process of switching of nodes needs to be done manually and there is no automatic failover mechanism in place.

- NameNode receives heartbeat signals and a Block Report periodically from each of the DataNo
- Heartbeat signals from a DataNode indicates that the corresponding DataNode is alive and is working fine.
- If a heartbeat signal is not received from a DataNode then that DataNode is marked as dead and no further I/O requests are sent to that DataNode.
- Block Report from each DataNode contains a list of all the blocks that are stored on that DataNo
- Heartbeat signals and Block Reports received from each DataNode help the NameNode to identify any potential loss of Blocks/Data and to replicate the Blocks to other active DataNodes in the event when one or more DataNodes holding the data blocks go down.


Client Application
Hadoop File System Client
NameNode
Sample.txt
Block A
Block B
Block C
Node A
Node B
Node C
DataNode A
A
B
C
DataNode B
A
B
C
DataNode C
A
B
C

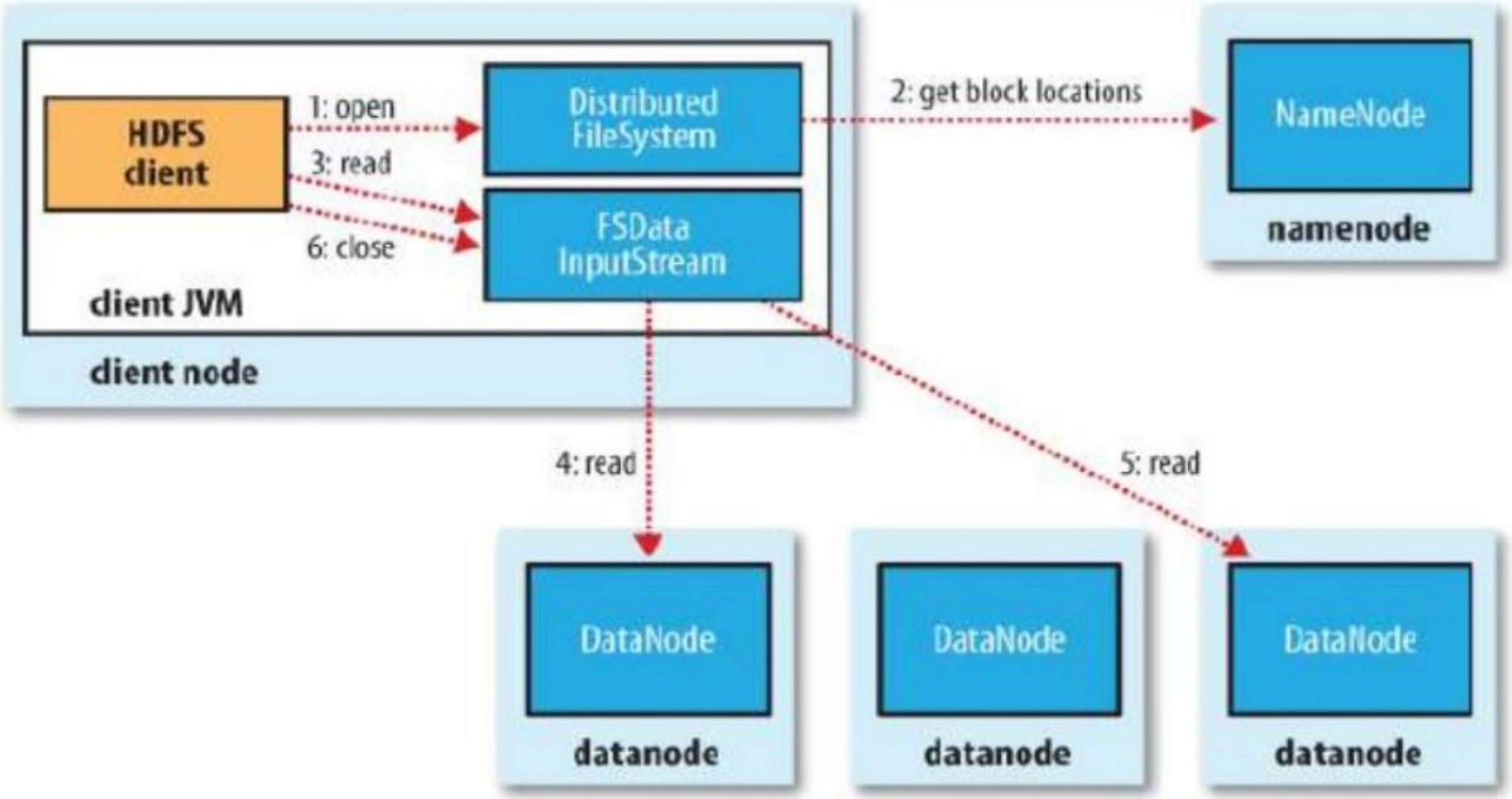
Anatomy of File Write in HDFS



Anatomy of a File Write in HDFS continue…

1. Client connects to the NameNode with file name.
2. The namenodeperforms various checks to make sure the file doesn't already exist, client has the right permissions etc
3. **NameNode places an entry for the file in its metadata,** returns the block name and list of DataNodes to the client.
4. **The list of datanodes forms a pipeline—we'll** assume the replication level is three, so there are three nodes in the pipeline.
5. Client connects to the first DataNode and starts sending data, As data is received by the first DataNode, it connects to the second and starts sending data Second DataNode similarly the second datanode stores the packet and forwards it to the third datanode in the pipeline.
6. *A packet is removed from the* ***ack*** **queue** only when it has been acknowledged by all the datanodes in the pipeline.
7. A **datanode fails** while data is being written to it, partial block on the failed datanode will be deleted if failed datanode recovers later on.
8. Client reports to the NameNode when the block is written.

Anatomy of a File Read in HDFS



1. Client **connects to the NameNode with file name.**
2. The namenode performs various checks to make sure the file exist, client has the right permissions etc ..
3. The namenode returns the addresses of the datanodes that have a copy of that block.(locality is considered)
4. The list of datanodes forms a pipeline—we'll assume the replication level is three, so there are three nodes in the pipeline.
5. The client connects to the first(closest) datanode for the first block in the file and reads. Then find the best datanode for the next block… and finish reads for the file.
6. **Verifies checksums for each block** the data transferred to it from the datanode.
7. **During reading, if the client encounters an error** while communicating with a datanode or block corrupted, then it will try the next closest one for that block.

Data processing in Hadoop(MapReduce Phases and Daemons)

Phases are :

1. Map : Converts input into Key-Value pairs

2. Reduce: Combines output of mappers and produces a reduced result set

There are 2 daemons asssociated with MapReduce Programming

1. A single master *JobTracker* per cluster

→Provides connectivity between Hadoop and your application

→When you submit the code to the cluster, JobTracker creates the execution plan by deciding which task to assign to which node.

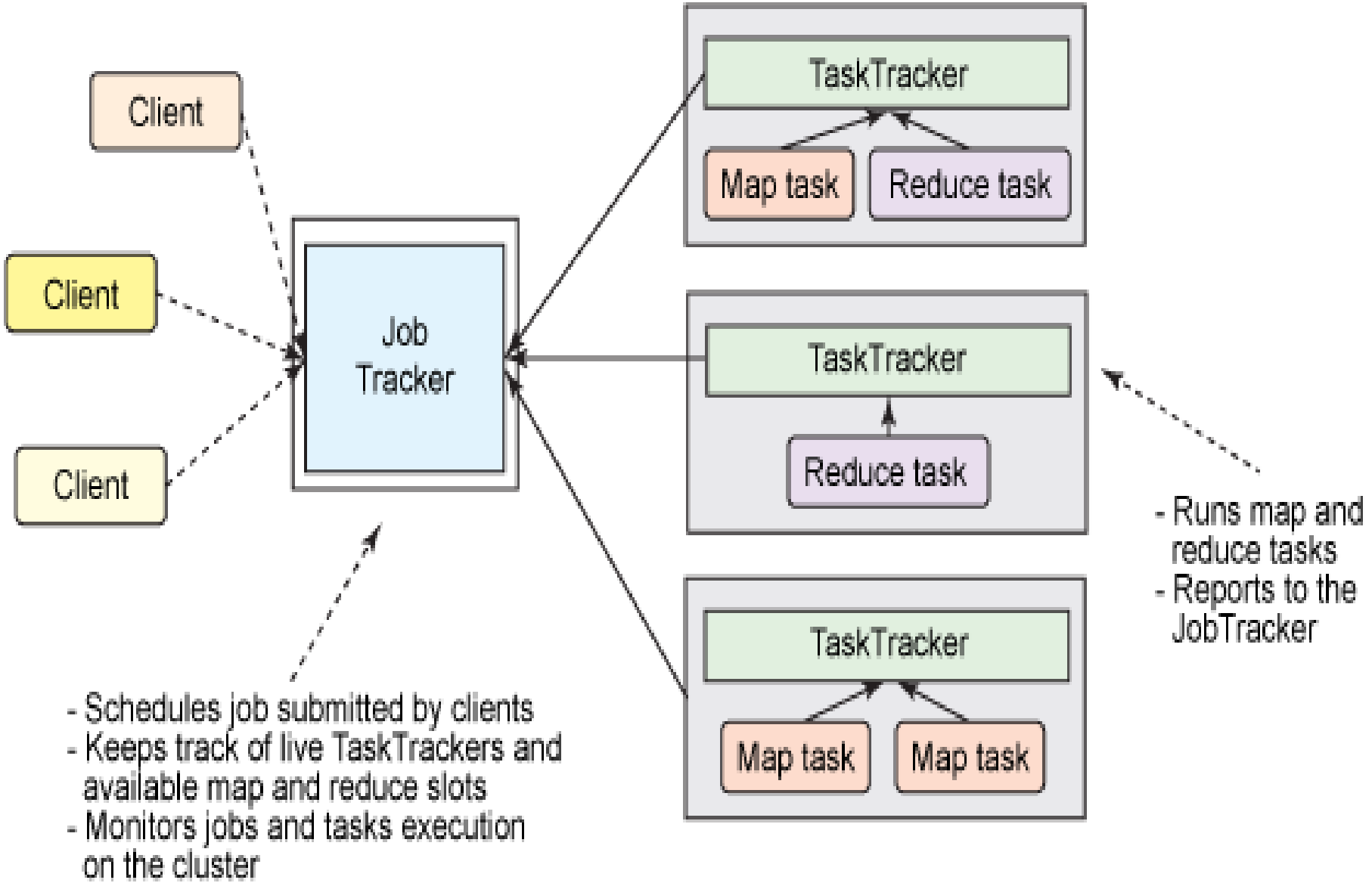
→It monitors all the running tasks.

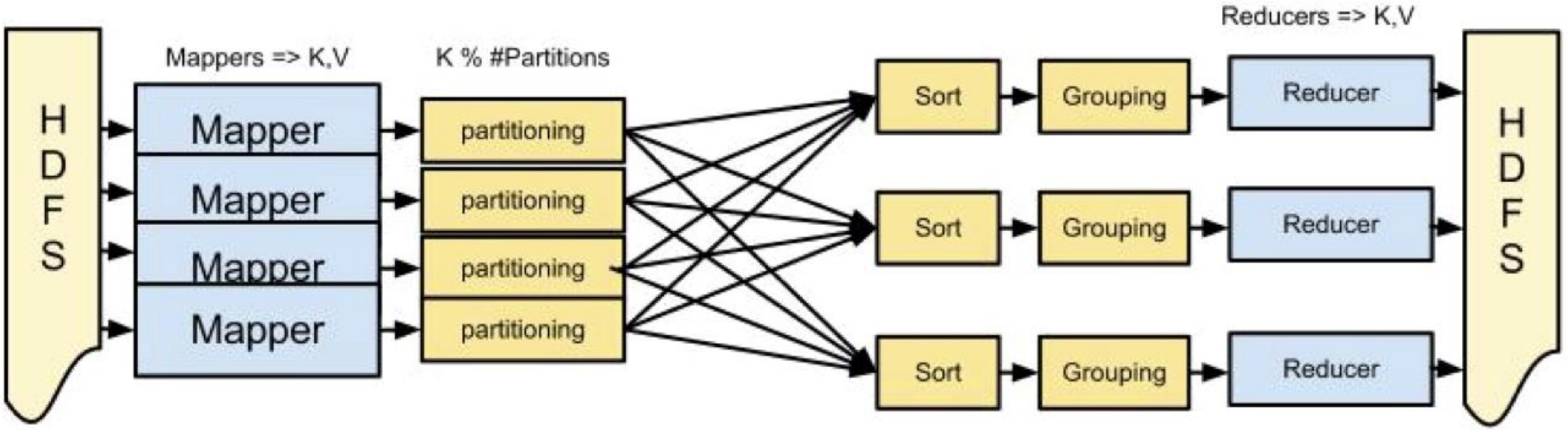
→When a task fails, it automatically re-schedules the task to a different node after a predefined

2. **One slave *TaskTracker* per cluster node.**

- This daemon is responsible for executing individual tasks that is assigned by the JobTracker.
- Handles multiple map or reduce tasks in parallel.
- TaskTracker continuously sends heartbeat messsage to JobTracker.
- When the JobTracker fails to recieve a heartbeat from a TaskTracker, the JobTracker assumes that the TaskTracker has failed and resubmits the task to another available node in the cluster.
- Once the client submits a job to the JobTracker, it partitions and assigns diverse MapReduce tasks for each TaskTracker in the cluster.

MapReduce Programming Architecture





The MapReduce Pipeline

A mapper receives (Key, Value) & outputs (Key, Value)
A reducer receives (Key, Iterable[Value]) and outputs (Key, Value)
Partitioning / Sorting / Grouping provides the Iterable[Value] & Scaling

MapReduce performs the following tasks:

1.The input data set is split into multiple copies of the data

2.The framework creates a master and several workers processes and executes the worker prosesses remotely

1.Several map tasks work simultaneously and read pieces of data that were assigned to each map task. The map worker uses the map function to extract only those data and generates key-value pair for extracted data

1.Map worker uses **partitioner** function to divide the data into regions. Partitioner decides which reducer should get the output of the specified mapper.

1.When the map workers complete their work, the master instructs the reduce workers to begin their work. The data thus recieved is shuffled and sorted as per keys.

1.Then it calls reduce function for every unique key. This function writes the output to the file.

2.When all the reduce workers complete their work, the master transfers the control to the user program.

MapReduce Example: Word count

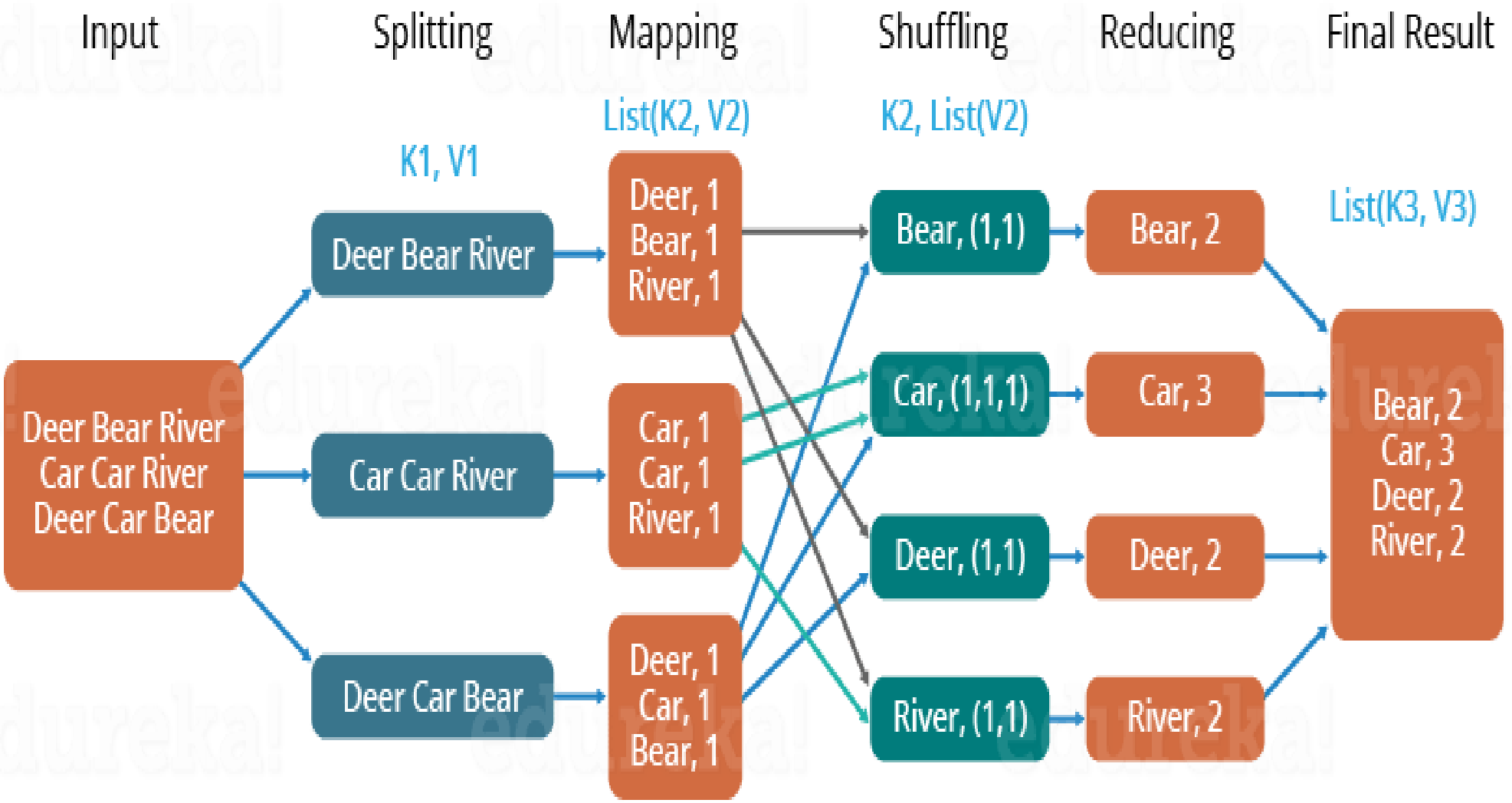
The MapReduce Programming requires 3 things

1.Driver Class: This class specifies Job Configuration Details

2.Mapper Class: This class overrides the Map Function based on the problem statement

3.Reducer Class: This class overrides the Reduce Function based on the problem statement

The Overall MapReduce Word Count Process



Limitations of Hadoop 1.x Architecture

1.Major drawback of Hadoop 1.x Architecture is **Single Point of Failure** as there is no backup Name Node.

1.Job scheduling, resource management and job monitoring are being done by Job Tracker which is tightly coupled with Hadoop. So Job Tracker is not able to manage resources outside Hadoop.

1.Job Tracker has to coordinate with all task tracker so in a very big cluster it will be difficult to manage huge number of task trackers altogether.

1.Due to single Name Node, there is no concept of namespaces in Hadoop 1.x. So everything is being managed under single namespace.

1.Using Hadoop 1.x architecture, Hadoop Cluster can be scaled upto ~4000 nodes. Scalablity

Limitations of Hadoop 1.x Architecture

6. It has a restricted processing model which is suitable for batch oriented

MapReduce jobs

7. Hadoop MapReduce is not suitable for interactivbe analysis

8. Hadoop 1.0 is not suitable for machine learning algorithms, graphs and other \

   memory intensive algorithms

HDFS 2

Consists of 2 major components

1. Namespace

2. Takes care of file related operations such as creating files, modifying files and directories

3.Blocks storage service

4. Handles data node cluster management, replication

HDFS 2 Features:

1. Horizontal Scalability

2.HDFS2 uses multiple independent namenodes which are independent of each other

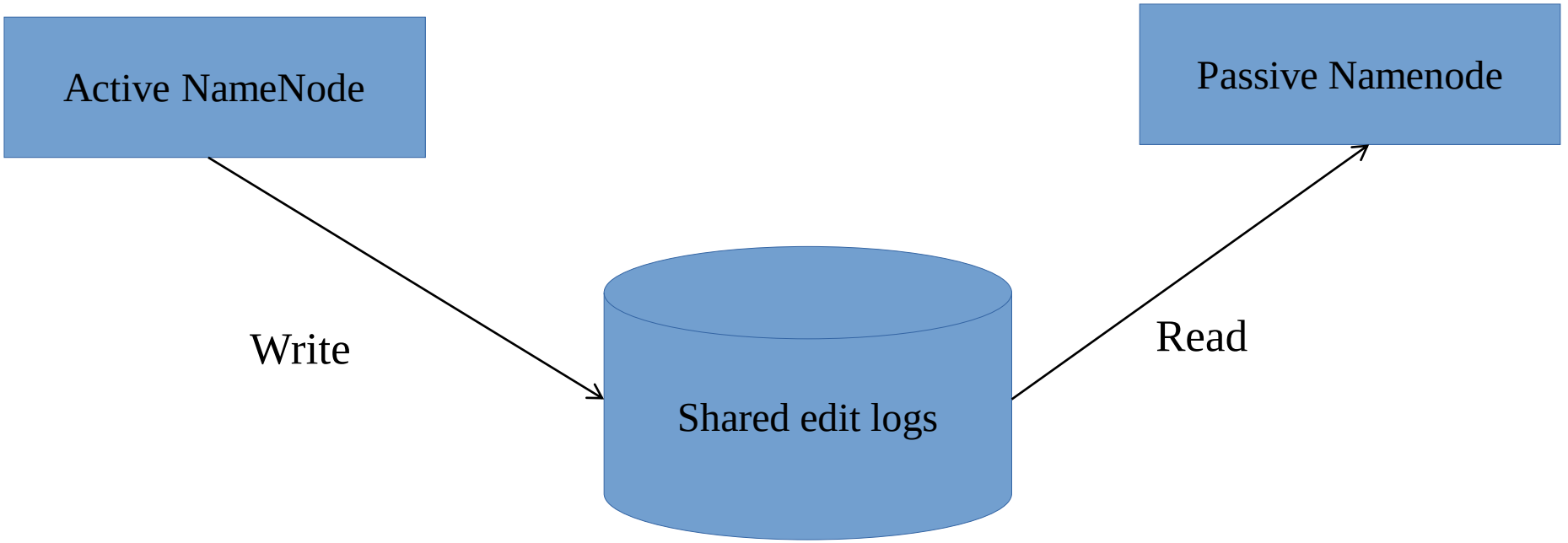
3.Data nodes are common storage for blocks and shared by all Name nodes

4.All the DataNodes in the cluster registers with each Namenode in the cluster.

5. High availability

6. It is obtained with the help of Passive standby NameNode

7. In Hadoop 2.x, Active-Passive NameNode handles failover automatically



All namespace edits are recorded to a shared storage and there is a single witer at any point of time

Passive NameNode reads edits from shared storage and keeps updated metadata information.

In case of Active Namenode failure, Passive Namenode becomes an Active Namenode automatically. Then it starts writing to the shared storage

YARN Architecture and Components

In Hadoop 1.x Architecture,

JobTracker daemon was carrying the responsibility of Job scheduling and Monitoring as well as was managing resource across the cluster.

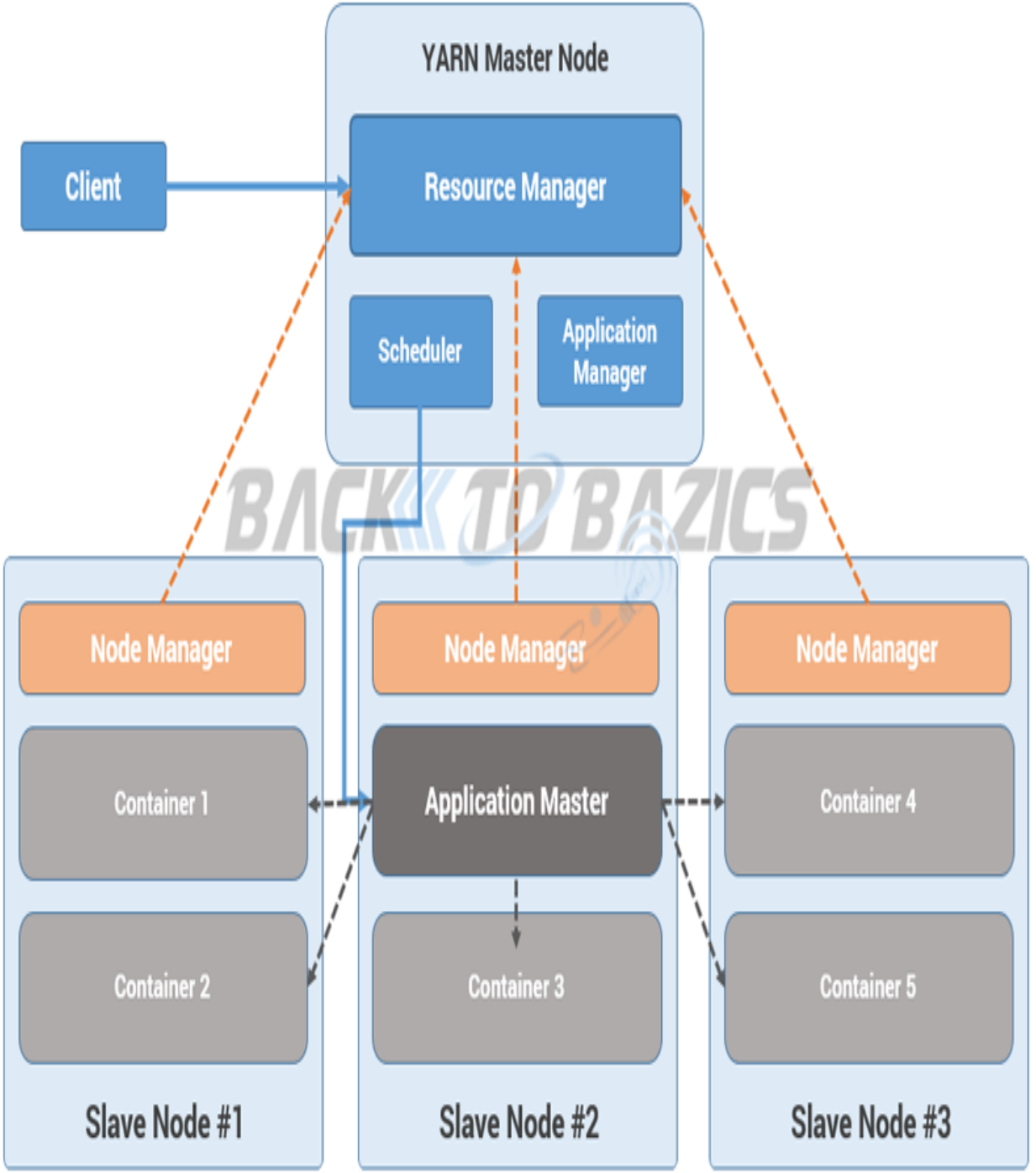
TaskTracker daemon was executing map reduce tasks on the slave nodes.

YARN has divided the responsibilities of JobTracker to two processes **ResourceManager and ApplicationMaster** and instead of TaskTracker is using **NodeManager** daemon for map reduce task execution.

YARN has total three major components

- **ResourceManager**

•**Client**: The user/application that submits a job.
•**YARN Master Node**:
•**Resource Manager**: Allocates resources across the cluster.
•**Scheduler**: Decides which node gets resources.
•**Application Manager**: Manages submitted jobs.
•**Slave Nodes**:
•**Node Manager**: Manages resources and containers on each node.
•**Containers**: Execution environments where tasks run.
•**Application Master**: Runs inside one of the containers, coordinates the job.



Real-time Example: Online Shopping Website (like Flipkart or Amazon)

Step-by-step:

Client → Submits Request

A customer searches for "iPhone 15".

The **client** sends this job (search request) to YARN.

Resource Manager → Handles Resources

The **Resource Manager** checks the cluster to see which nodes have free CPU and memory.

It decides where the search job should run.

Application Manager → Starts Application Master

The **Application Manager** launches an **Application Master** inside one of the nodes (say Slave Node #

This Application Master manages the search job execution.

Application Master → Requests Containers

The **Application Master** requests containers from Node Managers on different Slave Nodes.

Example:

Slave Node #1, Container 1 → Search product database

Slave Node #2, Container 3 → Filter results by price

Slave Node #3, Container 4 → Personalize results based on past user activity

Node Managers → Run Tasks

Each **Node Manager** allocates containers for the tasks assigned.

For example: one container searches the catalog, another applies filters, another handles recommer

Results Collected

The Application Master gathers results from all containers and sends them back to the client (your b

You now see a personalized list of iPhones on your screen.

1) ResourceManager

This daemon process resides on the Master Node (not necessarily on NameNode of Hadoop)

Responsible for,

- Managing resources scheduling for different compute applications in an optimum way

- Coordinating with two process on master node,

Scheduler and **ApplicationManager**

a. Scheduler

This daemon process resides on the **Master Node** (runs along with ResourceManager daemon)

Responsible for,

- Scheduling the job execution as per submission request received by ResourceManager
- Allocating resources to applications submitted to the cluster
- Coordinating with ApplicationManager daemon and keeping track of resources of running applications

b. ApplicationManager

This daemon process resides on the **Master Node** (runs along with ResourceManager daemon)

Responsible for,

- Helping Scheduler daemon to keeps track of running application by coordination
- Accepting job submissions from client
- Negotiating first container for executing application specific task with suitable ApplicationMaste

  on slave node

2) NodeManager

This daemon process resides on the slave nodes (runs along with DataNode daemon)

Responsible for,

- Managing and executing containers
- Monitoring resource usage (i.e. usage of memory, cpu, network etc..) and reporting it back to ResourceManager daemon
- Periodically sending heart-bits to ResourceManager for its health status update

3) ApplicationMaster

➢This daemon process runs on the slave node (along with the NodeManager daemon)

➢It is per application specific library works with **NodeManager** to execute the task

➢The instance of this daemon is per application, which means in case of multiple jobs submitted

➢on cluster, it may have more than one instances of ApplicationMaster on slave nodes

Responsible for,

- Negotiating suitable resource containers on slave node from ResourceManager
- Working with one or multiple NodeManagers to monitor task execution on slave nodes